# Crypto Trading Simulator

A Low-Latency, Hyper-Realistic Educational Sandbox




# System Overview

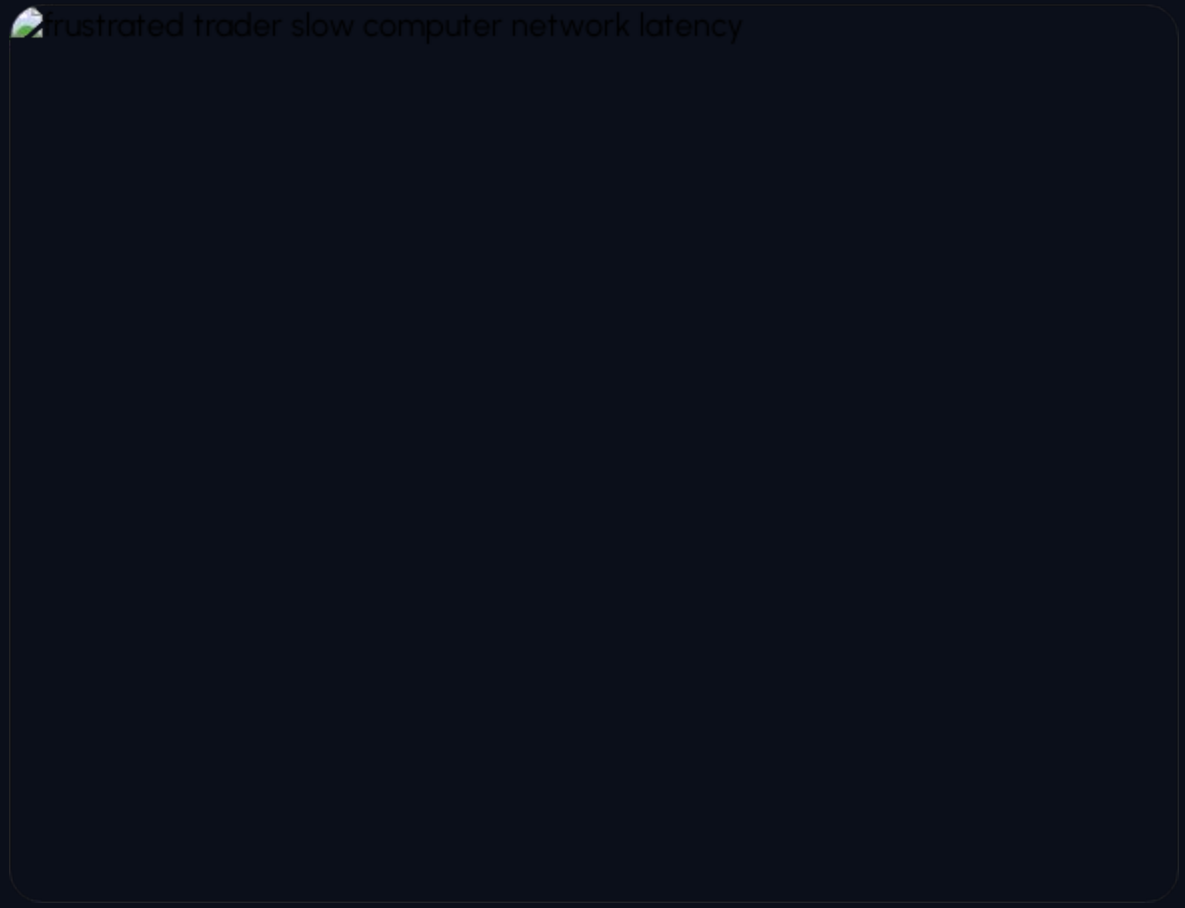
Traditional paper trading applications face significant challenges, including latency issues and paywalled access to advanced order types.

We propose a secure, low-latency Crypto Trading Simulator integrating a **FastAPI** asynchronous backend with live **Binance WebSocket** streams.

This ensures real-time, zero-delay price ticks for accurate matching engine execution, providing a professional-grade environment for testing trading strategies.

# The Problem

- **Slippage & Latency:** Current HTTP polling methods are inadequate, causing simulators to execute trades at prices the market has already moved past.
- **Lack of Advanced Mechanics:** Traditional simulators struggle to process dynamically changing variables, omitting tools like Trailing Stops.
- **Concurrency Failures:** Poorly designed database logic in simulators can lead to race conditions where users "double-spend" virtual funds.



# Core Objectives

## Accurate Execution

Reliable order triggering using live, sub-second tick data from WebSocket integrations.

## Ledger Integrity

Tamper-proof and atomic virtual wallet transaction storage via PostgreSQL row-level locks.

## Complex Orders

Free access to advanced conditional logic like Trailing Stops and Stop Limits.

## Professional UI

High-performance ReactJS charting without compromising application execution speed.

# Technology Stack

## Frontend Interface

- **ReactJS:** Dynamic, component-based UI for real-time portfolio tracking.
- **Tailwind CSS:** Utility-first styling for a highly responsive trading dashboard.
- **TradingView Charts:** Lightweight HTML5 canvas library for rendering high-performance financial visualizations smoothly.

## Core Execution Engine

- **FastAPI:** High-performance Python framework for routing and asynchronous logic.
- **Binance WebSockets:** Unauthenticated, continuous stream of live tick data for BTC, ETH, SOL, and SHIB.
- **Asyncio Matcher:** Background loops autonomously executing trades when conditions are met.

# Database & Security

**PostgreSQL** acts as the secure repository for storing virtual wallet balances, order history, and credentials.

To prevent race conditions and double-spending during rapid market volatility, the system implements **strict row-level locking**.

Additionally, **Redis** is utilized as an in-memory data store specialized for capturing and evaluating dynamically moving Trailing Stop thresholds before persisting them.



# Project Phases

## Phase 1

System Architecture & UI/UX Data flow design.

## Phase 2

FastAPI endpoints & Async matching loops.

## Phase 3

ReactJS dashboard & Charting implementation.

## Phase 4

Pytest validation & Concurrency stress testing.

# System Requirements

| Component | Hardware Specification | Software Requirement |
| --- | --- | --- |
| **Core System** | Intel Core i5-8250U @ 1.60GHz+ | Linux, Windows 10/11 |
| **Memory & Storage** | 8 GB RAM / 10 GB Free Space | PostgreSQL, SQLite |
| **Development** | Standard peripherals | Python, JavaScript/TypeScript |
| **Client Access** | Stable Internet Connection | Chrome, Edge, Safari, Firefox |

# Questions?

Ready to test the live market simulation?

# Image Sources



https://images.stockcake.com/public/1/8/3/1839be8c-00b1-4d1b-90c8-0a38710637a4_large/digital-trading-display-stockcake.jpg

Source: stockcake.com

---



https://cdn-ildomhl.nitrocdn.com/cUsQFJscahROvIBgFXJSfUSNtBfBxrhO/assets/images/optimized/rev-af06640/computercures.com.au/wp-content/uploads/2025/05/Computer-Slow-to-Boot.jpg

Source: computercures.com.au

---



https://png.pngtree.com/png-clipart/20250105/original/pngtree-high-tech-padlock-for-data-protection-png-image_18756528.png

Source: pngtree.com